

Esercizi specifiche algebriche

Metodi Avanzati di Programmazione
(Track M-Z)

Dipartimento di Informatica
Università degli Studi di Bari Aldo Moro

docente: prof. Pierpaolo Basile

Esercizio 1 (1/3)

Sia data la seguente specifica algebrica **StringSpec**

sorts: String, Char, Int, Bool;

operations

new() → String;

add(String, Char) → String;

append(String, String) → String;

length(String) → Int;

isEmpty(String) → Bool;

equal(String, String) → Bool;

Esercizio 1 (2/3)

declare: s, s1, s2: String, c: Char

```
isEmpty(new()) = true;  
isEmpty(add(s, c)) = false;  
length(new()) = 0;  
length(add(s, c)) = length(s) + 1;  
append(s, new()) = s;  
append(s1, add(s2, c)) = add(append(s1, s2), c);  
equal(new(), new()) = true;  
equal(new(), add(s, c)) = false;  
equal(add(s, c), new()) = false;  
equal(add(s1, c), add(s2, c)) = equal(s1, s2);
```

Esercizio 1 (3/3)

Dimostrare che:

`append(new(), add(new(), c)) = add(new(), c)`

Soluzioni Esercizio 1

Soluzione Esercizio 1

Dimostrare che:

`append(new(), add(new(), c)) = add(new(), c)`

=> utilizzando `append(s1, add(s2, c)) = add(append(s1, s2), c)`

`add(append(new(), new()), c) = add(new(), c)`

=> utilizzando `append(s, new()) = s` (`s=new()` => `append(new(), new()) = new()`)

`add(new(), c) = add(new(), c)`

Esercizio 2

Definire la specifica algebrica della struttura dati `Coord` che rappresenta una coordinata nello spazio cartesiano a due dimensioni.

sorts: `Coord`, `Integer`, `Boolean`

operations:

`Create(Integer, Integer) → Coord ;`

`X(Coord) → Integer ;`

`Y(Coord) → Integer ;`

`Eq(Coord, Coord) → Boolean ;`

Soluzione Esercizio 2

Soluzione Esercizio 2

Osservazioni	Costruttori di c'
	Create(x, y)
$X(c')$	x
$Y(c')$	y
$Eq(c', c)$	if $x=X(c)$ then if $y=Y(c)$ then true else false else false

Soluzione Esercizio 2

declare: x,y: Integer; c: Coord

$X(\text{create}(x, y)) = x$

$Y(\text{create}(x, y)) = y$

$\text{Eq}(\text{create}(x, y), c) = \text{if } x=X(c) \text{ then (if } y=Y(c) \text{ then true else false) else false}$

Esercizio 3

Realizzare la specifica algebrica della struttura dati albero binario utilizzando il seguente ordine di inserimento: se il nodo da inserire è minore del nodo corrente inserirlo nel sotto albero sinistro altrimenti inserirlo nel sotto albero destro. L'albero binario contiene dati generici di tipo Elem.

Soluzione Esercizio 3

Soluzione Esercizio 3

Realizzare la specifica algebrica della struttura dati albero binario utilizzando il seguente ordine di inserimento: se il nodo da inserire è minore del nodo corrente inserirlo nel sotto albero sinistro altrimenti inserirlo nel sotto albero destro. L'albero binario contiene dati generici di tipo Elem.

sorts: Binary_tree, Boolean, Elem

operations

Create() $\rightarrow$ Binary_tree

Add(Binary_tree, Elem) $\rightarrow$ Binary_tree

Left(Binary_tree) $\rightarrow$ Binary_tree

Data(Binary_tree) $\rightarrow$ Elem

Right(Binary_tree) $\rightarrow$ Binary_tree

Is_empty(Binary_tree) $\rightarrow$ Boolean

Contains(Binary_tree, Elem) $\rightarrow$ Boolean

Soluzione Esercizio 3

Per facilitare la realizzazione della specifica algebrica è necessario introdurre un nuovo costruttore:

$\text{Build}(\text{Binary_tree}, \text{Elem}, \text{Binary_tree}) \rightarrow \text{Binary_tree}$

Questo operatore costruisce un albero dato il valore del nodo (Elem) e i rispettivi sottoalberi sinistro e destro (*primo e terzo parametro dell'operatore Build*)

I costruttori nel nostro caso sono gli operatori: Create e Build, tutti gli altri operatori sono degli osservatori.

Soluzione Esercizio 3

Osservatori	Costruttori di t'	
	Create()	Build(l, e, r)
Add(t', e')	Build(t', e', t')	if e' < e then Add(l, e') else Add(r, e')
Left(t')	Create()	l
Right(t')	Create()	r
Data(t')	error	e
IsEmpty(t')	true	false
Contains(t', e')	false	if e'=e then true else if (e'<e then Contains(l, e') else Contains(r, e')

Soluzione Esercizio 3

declare: l, r: Binary_tree, e, e': Elem

Add(Create(), e') = Build(Create(), e', Create())

Add(Build(l, e, r), e') = if $e' < e$ then Add(l, e') else Add(r, e')

Left(Create()) = Create()

Left(Build(l, e, r)) = l

Right(Create()) = Create()

Right(Build(l, e, r)) = r

Data(Create()) = error

Data(Build(l, e, r)) = e

IsEmpty(Create()) = true

IsEmpty(Build(l, e, r)) = false

Contains(Create(), e') = false

Contains(Build(l, e, r), e') = if $e'=e$ then true else (if $e'<e$ then Contains(l, e') else
Contains(r, e'))

Esercizio 4

Definire la specifica algebrica del tipo Cursore. Un Cursore è una rappresentazione di una posizione sullo schermo. Le operazioni definite sono:

- Create: associa ad un cursore un'icona in una specifica posizione dello schermo
- Position: restituisce la posizione corrente del cursore
- Translate: sposta il cursore andando a modificare le coordinate attuali di una certa quantità
- Change_Icon: che permette di cambiare l'icona associata al cursore

N.B. Come le immagini sono create sullo schermo e come viene associata un'immagine (icona) al cursore sono dettagli implementativi che non inseriamo nella specifica.

Soluzione Esercizio 4

Soluzione Esercizio 4

sorts: Cursor, Int, **Coord**, **Bitmap**

operations

Create(Coord, Bitmap) → Cursor

Translate(Cursor, Integer, Integer) → Cursor

Position(Cursor) → Coord

Change_Icon(Cursor, Bitmap) → Cursor

Dispaly(Cursor) → Cursor *// non considereremo questo operatore poiché dipende da dettagli implementativi*

Il tipo Bitmap rappresenta un'immagine su schermo, nel nostro caso l'icona.

E' possibile importare in una specifica una specifica precedentemente definita, in questo caso **Coord**.

Soluzione Esercizio 4

Osservatori	costruttore di c'
	Create(d, b)
Translate(c', xi, yi)	Create(Coord.Create(Coord.X(d)+xi, Coord.Y(d)+yi), b)
Position(c')	d
Change_Icon(c', b')	Create(d, b')

d è di tipo Coord (Esercizio 2)

Soluzione Esercizio 4

declare: d: Coord, b, b': Bitmap

Translate(Create(d, b), xi, yi) = Create(Coord.Create(Coord.X(d)+xi, Coord.Y(d)+yi), b)

Position(Create(d, b)) = d

Change_Icon(Create(d, b), b') = Create(d, b')